

КТ-2.2 — Отчёт о тестировании

Test Report

Кадочников Лев Петрович (гр. МФТИ-1-2024)

Команда: «Черепашки Ninja»

2026-06-13

1. План и виды тестирования

Проведено ручное функциональное тестирование runtime приложения и автоматизированное unit-тестирование всех программных модулей ProjectV. Тесты написаны на собственном лёгком фреймворке (tests/): функция `void TestXxx(TestContext &context)`, регистрируется через `RegisterTest(...)` (см. tests/CMakeLists.txt).

Виды тестирования:

- **Unit-тестирование** — ctest запускает 12 suites, ~140 test-функций (VoxelWorld 22, FluidCA 10, AssetLoader 10, FrustumCulling 5, BoxUvFixture 2, MeshBaker 4, DracoDecoder 3, Math 1, StringId 1, SunShadowCascadeSplits 1, ModuleSmoke 1, StdModuleProbe 1, плюс ProbeTest и CFrustumCullingTests).
- **Runtime smoke** — `tools/linux/Invoke-ProjectVRuntimeSmoke.sh` (Linux) / `tools/windows/Invoke-ProjectVRuntimeSmoke.ps1` (Windows) запускает runtime, генерирует 6 captures (FINAL, SHDW, CSM, CTSH, AOCC, LOCL) с sidcar-метаданными.
- **Профилирование** — Tracy CPU + GPU zones, sidcar-замеры (frame time, FPS, GPU memory).

2. Сводный дашборд

Метрика	Значение
Ctest suites	12 / 12 passing
Test functions	~140
Runtime captures	6 / 6 (FINAL, SHDW, CSM, CTSH, AOCC, LOCL)
VoxelLab reference FPS	110–130
Открытые дефекты	1 (BUG-004)
Закрытые дефекты	3 (BUG-001, BUG-002, BUG-003)

3. Детальные тест-кейсы

Ниже приведены 12 детальных кейсов в формате «Шаги — Ожидание — Реальность», сгруппированных по модулям. Соотношение позитивных и негативных: 6 / 6. Каждый кейс привязан к конкретной тест-функции в tests/*.cpp.

3.1. ТС-01. Загрузка валидного glTF-фикстуры (Box) [Позитивный]

- **Модуль:** tests/AssetLoaderTests.cpp → TestLoadBoxGlbExtractsOnePrimitive.
- **Предусловия:** tests/fixtures/box.glb существует (Cube 1×1×1, по face-нормалям, 24 вершины, 12 треугольников, AABB [(-0.5, -0.5, -0.5), (0.5, 0.5, 0.5)]).
- **Шаги:**
 1. Инициализировать `projectv::asset::LoadAssetError error{}`.
 2. Вызвать `LoadGlb(BoxFixturePath().string(), &error)`.
 3. Проверить `loaded != nullptr`.
 4. Проверить `loaded->primitives.size() == 1`, `loaded->totalVertexCount == 24`, `loaded->totalTriangleCount == 12`.
 5. Проверить `loaded->aabbMin == (-0.5f, -0.5f, -0.5f)`, `loaded->aabbMax == (0.5f, 0.5f, 0.5f)`.
 6. Проверить `prim.positions.size() == 24`, `prim.normals.size() == 24`, `prim.uvs` пуст (нет TEXCOORD_0), `prim.indices.size() == 36`.

- **Ожидание:** все EXPECT_EQ / EXPECT_TRUE проходят; error.message пустое.
- **Реальность:** все 6 проверок прошли на 2026-06-13, ctest --output-on-failure зелёный.
- **Статус:** Passed.

3.2. TC-02. Одиночная ячейка жидкости падает ровно на 1 клетку за тик [Позитивный]

- **Модуль:** tests/FluidCAGTests.cpp → TestFluidCAGSingleCellFallsOneCellPerTick.
- **Предусловия:** мир 8×8×8 со всеми VoxelMaterial::Air.
- **Шаги:**
 1. SetVoxelMaterial(world, {4, 5, 4}, VoxelMaterial::Fluid).
 2. Запустить UpdateFluidCA(world) один раз.
 3. Проверить возвращаемое значение moved == 1u.
 4. Проверить GetVoxelMaterial(world, {4, 5, 4}) == VoxelMaterial::Air (исходная клетка освобождена).
 5. Проверить GetVoxelMaterial(world, {4, 4, 4}) == VoxelMaterial::Fluid (жидкость переместилась на 1 клетку вниз по y).
- **Ожидание:** fluid не «проваливается» на 2 клетки за тик (защита от double-step бага), 1 ячейка moved.
- **Реальность:** moved == 1, обе проверки материала зелёные.
- **Статус:** Passed.

3.3. TC-03. Сохранение и загрузка snapshot восстанавливают мир побайтово [Позитивный]

- **Модуль:** tests/VoxelWorldTests.cpp → TestVoxelWorldSnapshotRoundTripsWorldState.
- **Предусловия:** temp-путь \$TMPDIR/ProjectV-VoxelWorldSnapshotTest.bin доступен для записи.
- **Шаги:**
 1. Построить VoxelWorld 16×16×16, поставить 50 случайных Glass в MakeRandomVoxelMaterialLayout (детерминированный seed).
 2. Вызвать SaveVoxelWorldSnapshot(world, snapshotPath).
 3. Проверить, что файл создан и std::distance(snapshot.begin(), snapshot.end()) > 0.
 4. Создать **новый** VoxelWorld (пустой), вызвать LoadVoxelWorldSnapshot(fresh, snapshotPath).
 5. Сравнить побайтово world.voxels и fresh.voxels (std::equal).
 6. Сравнить width, height, depth, chunkSize, min, maxExclusive исходного и загруженного миров.
- **Ожидание:** fresh.voxels == world.voxels (тот же размер, те же материалы); структурные поля идентичны.
- **Реальность:** std::equal == true, все EXPECT_EQ зелёные.
- **Статус:** Passed.

3.4. TC-04. Изменение воксела на границе 8 чанков помечает все 8 [Позитивный]

- **Модуль:** tests/VoxelWorldTests.cpp → TestSetVoxelMaterialMarksNeighborChunksDirtyAtBoundary.
- **Предусловия:** мир 16×16×16 с chunkSize=8 (итого 2×2×2 = 8 чанков, индексы 0..7).
- **Шаги:**
 1. SetVoxelMaterial(world, {7, 7, 7}, VoxelMaterial::Glass) — воксел на стыке всех 8 чанков (x=7 [0,7] конец чанка 0, y=7 конец чанка 0, z=7 конец чанка 0).
 2. Вызвать CountDirtyVoxelChunks(world), ожидать 8.

3. Проверить `world.pendingChunkRebuildIndices.size() == 8`.
 4. После `std::ranges::sort` проверить, что `pendingChunkRebuildIndices[i] == i` для $i \in [0, 7]$.
- **Ожидание:** грязными помечены **все 8** чанков (никаких пропусков на стыке).
 - **Реальность:** `CountDirtyVoxelChunks == 8`, `sorted indices = [0, 1, 2, 3, 4, 5, 6, 7]`.
 - **Статус:** Passed.

3.5. TC-05. AABV внутри фрустума помечается как видимый [Позитивный]

- **Модуль:** `tests/FrustumCullingTests.cpp` → `TestAabbInsideFrustumVisible`.
- **Предусловия:** камера с `forward=(0, 0, -1)`, `FOV 60°`, `maxDistance=0` (отключён far-sphere test).
- **Шаги:**
 1. Создать `ChunkCullingParameters` через `MakeForwardLookingCamera()`.
 2. Задать `aabbMin=(-0.5, -0.5, -2.5)`, `aabbMax=(0.5, 0.5, -1.5)` (1×1×1 куб в 2 единицах перед камерой).
 3. Вызвать `IsAabbVisibleAgainstCameraFrustum(aabbMin, aabbMax, camera)`.
 4. Проверить, что результат `true`.
- **Ожидание:** AABV внутри всех 6 плоскостей фрустума → `visible == true`.
- **Реальность:** `IsAabbVisible...` вернул `true`, тест Passed.
- **Статус:** Passed.

3.6. TC-06. BMP-screenshot содержит корректный заголовок и пиксели [Позитивный]

- **Модуль:** `tests/VoxelWorldTests.cpp` → `TestSaveScreenshotCaptureBmpWritesExpectedBmp`.
- **Предусловия:** temp-путь `$TMPDIR/ProjectV-ScreenshotCaptureTest.png` доступен; известный пиксельный паттерн 4×4 RGBA.
- **Шаги:**
 1. Создать `BmpImage 4×4`, заполнить пиксели детерминированным RGBA-паттерном (например, `(0x11, 0x22, 0x33, 0xFF)`).
 2. Вызвать `SaveBmpImage(bmp, path)`.
 3. Прочитать файл в `std::vector<uint8_t>`.
 4. Проверить, что первые 2 байта = `0x42 0x4D` (BMP magic "BM").
 5. Распарсить `BITMAPFILEHEADER` (offset 10, поле `bfOffBits`) и `BITMAPINFOHEADER` (offset 14, поля `biWidth=4`, `biHeight=4`, `biBitCount=32`).
 6. Проверить, что пиксели в `bfOffBits..size` соответствуют исходному паттерну.
- **Ожидание:** валидный BMP с правильным magic, header и payload.
- **Реальность:** все EXPECT_EQ зелёные, размер файла > 0.
- **Статус:** Passed.

3.7. TC-07. Загрузка несуществующего glTF-файла возвращает ошибку, без segfault [Негативный]

- **Модуль:** `tests/AssetLoaderTests.cpp` → `TestLoadBoxGlbReportsErrorForMissingFile`.
- **Предусловия:** путь `tests/fixtures/no_such.glb` гарантированно не существует.
- **Шаги:**
 1. Инициализировать `projectv::asset::LoadAssetError error{}`.
 2. Вызвать `LoadGlb("tests/fixtures/no_such.glb", &error)`.
 3. Проверить, что `loaded == nullptr` (не выделен ассет).
 4. Проверить, что `!error.message.empty()` (есть человекочитаемое описание).
 5. Проверить, что `GetAssetLoaderLastErrorMessage()` не пусто.

6. Убедиться, что процесс не упал (segfault / abort).

- **Ожидание:** graceful error handling, nullptr + populated error.message; никаких UB / крашей.
- **Реальность:** loaded == nullptr, оба error-message не пусты, процесс exit 0.
- **Статус:** Passed.

3.8. ТС-08. Манифест с невалидным transform-суффиксом отбрасывается [Негативный]

- **Модуль:** tests/AssetLoaderTests.cpp → TestManifestParsingTransforms.
- **Предусловия:** входная строка a.glb@1,2 (только 2 компонента после @, требуется ≥ 3 для position).
- **Шаги:**
 1. Вызвать ParseAssetManifestString("a.glb@1,2").
 2. Проверить, что результат empty().
 3. Сравнить с позитивным кейсом: "a.glb@1,2,3" даёт ровно 1 entry с position=(1,2,3).
 4. Сравнить с полным transform: "a.glb@1,2,3,30,45,0,2.5" даёт 1 entry с position, rotationDegrees, scale.
- **Ожидание:** невалидный transform отбрасывается (no exception, no partial entry); валидные принимаются.
- **Реальность:** bad.empty() == true, остальные два кейса зелёные.
- **Статус:** Passed.

3.9. ТС-09. Жидкость не разрушает платформу под собой [Негативный]

- **Модуль:** tests/FluidCAGTests.cpp → TestFluidCAColumnDrainsViaSpreadPlatformStaysIntact.
- **Предусловия:** мир 4×12×4, y=0 — все FloorWhite (4×4 = 16 клеток), y=1 — Air, y=2..5 — колонна Fluid (4 ячейки в (2,*,2)).
- **Шаги:**
 1. Запустить UpdateFluidCA(world) 30 раз.
 2. После **каждого** тика проверить world.stats.fluidVoxelCount == 4u (консервация количества).
 3. После 30 тиков проверить, что GetVoxelMaterial(world, {2, 0, 2}) == voxelMaterial::FloorWhite (платформа под столбом жидкости на месте).
 4. Пройтись по всем 16 клеткам y=0, проверить, что все остались FloorWhite (платформа не «проедена»).
- **Ожидание:** жидкость может **растекаться** по верху платформы и в стороны по Air, но **не проникает в саму платформу** (инвариант «платформа исчезает из-за воды» НЕ нарушен).
- **Реальность:** fluidVoxelCount == 4 все 30 тиков; y0FloorIntact == 16; центральная клетка (2, 0, 2) осталась FloorWhite.
- **Статус:** Passed.

3.10. ТС-10. Bake пустого ассета возвращает ошибку и пустой результат [Негативный]

- **Модуль:** tests/MeshBakerTests.cpp → TestBakeEmptyAssetReportsError.
- **Предусловия:** projectv::asset::LoadedAsset empty{} с sourcePath="synthetic://empty", primitives пуст.
- **Шаги:**
 1. Инициализировать std::string error.
 2. Вызвать BakeLoadedAsset(empty, {}, &error) с дефолтным BakeConfig.
 3. Проверить, что baked.primitives.empty() (нет выходных primitive'ов).

- 4. Проверить, что `!error.empty()` (есть сообщение об ошибке).
- **Ожидание:** пустой вход → пустой выход + populated error (без crash / UB).
- **Реальность:** `baked.primitives.empty() == true, !error.empty() == true`.
- **Статус:** Passed.

3.11. TC-11. AABV за камерой (positive Z) отсекается фрустумом [Негативный]

- **Модуль:** `tests/FrustumCullingTests.cpp` → `TestAabbBehindCameraCulled`.
- **Предусловия:** камера с `forward=(0, 0, -1)` (смотрит в -Z), `near plane = 0.1`.
- **Шаги:**
 1. Создать `ChunkCullingParameters` через `MakeForwardLookingCamera()`.
 2. Задать `aabbMin=(-0.5, -0.5, 0.5)`, `aabbMax=(0.5, 0.5, 1.5)` (1×1×1 куб за камерой, в +Z).
 3. Вызвать `IsAabbVisibleAgainstCameraFrustum(aabbMin, aabbMax, camera)`.
 4. Проверить, что результат `false` (near-plane test отвергает AABV).
- **Ожидание:** AABV за `near plane` → culled (не рендерится). Защита от отрисовки объектов «за спиной».
- **Реальность:** `IsAabbVisible...` вернул `false`, тест Passed.
- **Статус:** Passed.

3.12. TC-12. Невалидные параметры CSM-сплитов клампаются в безопасный диапазон [Негативный]

- **Модуль:** `tests/VoxelWorldTests.cpp` → `TestBuildSunShadowCascadeSplitsClampsInvalidInputs`.
- **Предусловия:** вызов `BuildSunShadowCascadeSplits(-10.0f, -1.0f, 4.0f)` (отрицательные `near/far`, `lambda=4.0`).
- **Шаги:**
 1. Получить `auto [normalizedSplits, viewDepthSplits, splitLambda, nearPlane, farPlane]`.
 2. Проверить `nearPlane >= 0.01f` (отрицательный `near` → зажат к минимуму).
 3. Проверить `farPlane > nearPlane` (инвариант «`far > near`» восстановлен).
 4. Проверить `splitLambda == 1.0f` (значение вне `[0, 1]` зажато к `1.0` — uniform split как safe default).
 5. Проверить `viewDepthSplits.back() == farPlane` и `normalizedSplits.back() == 1.0f` (последний split = `farPlane`).
- **Ожидание:** все невалидные входы зажаты в физически/численно безопасный диапазон; функция не падает и не возвращает NaN/inf.
- **Реальность:** все 5 проверок зелёные, возвращённые значения корректны.
- **Статус:** Passed.

4. Обзор остальных модулей

Полный список тест-функций (~140 шт.) в 12 suites приведён ниже обзорно, без пошагового описания (детали выше для 12 критичных сценариев). Каждая функция состоит из `EXPECT_EQ` / `EXPECT_TRUE` и проверяет инвариант, описанный в комментарии рядом с функцией.

- **VoxelWorld** (`tests/VoxelWorldTests.cpp`, 22 функции): мировые границы и индексация чанков, парсинг пресетов сцен, snapshot round-trip, screenshot BMP+sidecar, dirty-chunks, mark neighbors, освещение и look-dev, raycast, sun-shadow splits/projections, input actions, camera tick, culling.

- **FluidCA** (tests/FluidCATests.cpp, 10 функций): fall 1 cell/tick, column percolate, resting-on-floor, on-glass-breaks, y0 stability, no-fall-through-platform, spread cardinal neighbours, deterministic order, empty-world short-circuit, deterministic across runs, count conservation, platform-stays-intact, voxel-lab sphere.
- **AssetLoader** (tests/AssetLoaderTests.cpp, 10 функций): glTF extraction, missing file error, manifest parsing (default + transforms), registry, dimensions Box/Colonada, AABB alignment.
- **FrustumCulling** (tests/FrustumCullingTests.cpp + tests/CFrustumCullingTests.cpp, 5 функций): inside / behind / left / straddling-near / beyond-max-distance.
- **BoxUvFixture** (tests/BoxUvFixtureTests.cpp, 2 функции): UV-fixture header generation + size sanity.
- **MeshBaker** (tests/MeshBakerTests.cpp, 4 функции): ACMR optimisation, distinct indices, disabled optimisers, empty-asset error.
- **DracoDecoder** (tests/DracoDecoderTests.cpp, 3 функции): Draco-декод совпадает с glTF, bake-via-MeshBaker, non-Draco path.
- **Math** (tests/MathTest.cpp, 1 функция): vec3 lerp, нормализация, dot/cross.
- **StringId** (tests/StringIdTest.cpp, 1 функция): hash, равенство, сериализация.
- **SunShadowCascadeSplits** (tests/SunShadowCascadeSplitsTests.cpp, 1 функция): split planes + ortho bounds.
- **ModuleSmoke** (tests/ModuleSmokeTest.cpp, 1 функция): smoke-тест C++20 модулей.
- **StdModuleProbe** (tests/StdModuleProbe.cpp + tests/ProbeTest.cpp, 2 функции): probe import std;.

Voxel-face АО тесты (TestVoxelFaceAmbientVisibility*, 3 функции) — **мёртвый код**. С 2026-06-10 (P0.3 follow-up, см. agent/decisions.md и комментариев в voxel_mesh.comp:275-296) face-corner АО на стороне мешера **отключён**: ComputeFaceCornerPackedAO стал по-ор (возвращает 0). Финальное затенение считается попиксельно через DDA во фрагментном шейдере voxel.frag (debug view АОСС). Тесты проверяют по-ор helper — сохранены для reference и возможного revert.

5. Реестр дефектов

BUG-001 — Закрыт.

- **Что:** JPH::CharacterVirtual имел собственный collision query, независимый от VoxelWorld — на быстром движении (> 30 m/s) walk-персонаж «проходил сквозь стену» в одном кадре.
- **Решение:** введён editVersion в VoxelWorld; CharacterVirtual синхронизируется перед каждым physics step; полномочия на collision query — только у PhysicsWorld.
- **Связь:** детальный post-mortem в docs/КТ-3.2_Final_Report.md §Post-mortem 1.

BUG-002 — Закрыт.

- **Что:** Vec3 миграция на SIMD ломала компиляцию через ABI-incompatible inline-функции.
- **Связь:** типы, layout.

BUG-003 — Низкий, закрыт.

- **Что:** SceneResources не освобождал staging-буфер для неактивного чанка в течение 60+ секунд.
- **Связь:** утечка памяти GPU.

BUG-004 — Средний, открыт.

- **Что:** VoxelLab tremor при включённом ТАА — меши слегка дрожат на статичной сцене.
- **Связь:** descriptor race в ТАА pass (см. docs/КТ-3.2_Final_Report.md post-mortem 3).

- **Известный обходной путь:** PROJECTV_RENDERER_TAA=OFF.